Free LLM Gateway Setup Guide

Stacking free provider tiers behind one OpenAI-compatible endpoint — FreeLLMAPI and OmniRoute

---

Both projects solve the same problem. Every AI lab offers a free tier; each one alone is a toy that dies halfway through a single agent run. But the limits are **independent** — Groq rate-limiting you says nothing about Cerebras. If you treat all of them as one logical pool and move to the next provider the instant one returns 429, your effective ceiling becomes the sum rather than the maximum. Everything else in these tools is plumbing around that one idea.

	OmniRoute	FreeLLMAPI
Repo	github.com/diegosouzapw/OmniRoute	github.com/tashfeenahmed/freellmapi
Scale	352 providers, ~1,300 model IDs	34 providers, 635 endpoints
Default port	20128	3001
Install	npm i -g omniroute	Docker / one-line script
Licence	MIT	MIT (paid tier = live catalog feed only)
Character	Feature-dense, large surface area	Narrow, cleaner, better docs

Read this before you start

Neither tool creates accounts for you and there is no shared pool of credentials. You sign up at each provider yourself, accept each ToS yourself, and paste your own keys in. The headline token figures assume you hold a key for every provider in the catalog. Run this on a development machine only — never in front of a production deployment, and never with client or work data.

Step 0 · Get provider keys (shared by both tools)

This is the part that actually takes time, and it is identical either way. Do not attempt all 34. Start with five that give the most usable capacity for agent work:

Provider	Console	Why it earns a slot
Google AI Studio	aistudio.google.com	Gemini Flash; large context, generous daily requests
Groq	console.groq.com	Fastest inference by a wide margin — ideal for agent loops
Cerebras	cloud.cerebras.ai	Very fast, decent daily cap, narrow model list
Mistral	console.mistral.ai	Monthly token budget, solid tool calling
OpenRouter	openrouter.ai	An aggregator itself — one key unlocks a long tail of free models

Optional second wave: Cloudflare Workers AI, NVIDIA NIM (build.nvidia.com), Z.ai / Zhipu, Cohere, HuggingFace. **Skip ModelScope** — it requires an Aliyun China identity binding.

One account per provider. Farming five Groq accounts to multiply your quota is where stacking crosses into abuse and gets keys banned. The tools will not stop you; the providers will.

FreeLLMAPI · recommended starting point

github.com/tashfeenahmed/freellmapi

Install

The README offers a one-liner that pipes a remote script to bash. Read it first at `freellmapi.co/install.sh`, or skip it and drive Compose directly — which is what the script sets up anyway.

```
# Option A - the one-liner (Docker required)
curl -fsSL https://freellmapi.co/install.sh | bash

# Option B - clone and run Compose yourself
git clone https://github.com/tashfeenahmed/freellmapi.git
cd freellmapi
cp .env.example .env
# generate an encryption key into .env, per docs/en/install/01-install.md
docker compose up -d
```

Option A creates `~/freellmapi`, generates the encryption key, pulls the image and starts the container. Re-running it is safe: your `.env` and encryption key are preserved and the container updates to latest. On Windows the desktop .exe from Releases is the easiest path; a macOS .dmg exists too, and the desktop build signs itself in with a hidden local account, so there is no password to set up.

Configure

- Open `http://localhost:3001`. Server installs create an email + password account. If you forget it, the reset code prints to the server log — `docker compose logs -f freellmapi`.
- **Keys page** — paste each provider key. Each gets a status dot and a health-check timestamp. Keys are AES-256-GCM encrypted in local SQLite and decrypted in memory per request.
- **Models page** — reorder the fallback chain. Groq and Cerebras high if you care about latency; Gemini high if you care about context length.
- Copy the unified `freellmapi-…` key from the Keys page header. That single key is all your application ever sees.

Use from Python

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:3001/v1",
    api_key="freellmapi-your-unified-key",
)

resp = client.chat.completions.create(
    model="auto",   # or "auto:fast", "auto:smart", a profile, or a model id
    messages=[{"role": "user", "content": "ping"}],
)
print(resp.choices[0].message.content)
```

Every response carries an `X-Routed-Via: <platform>/<model>` header naming the provider that actually served it. Log that header during development — it is how you will notice quality drift as daily caps are hit.

Wire up coding agents

```
npx freellmapi setup-claude --url http://localhost:3001 --api-key <unified-key>
npx freellmapi setup-codex
npx freellmapi setup-aider
```

Every generator supports `--dry-run`, writes a timestamped backup before touching an existing file, and merges rather than clobbering. For Claude Code and Codex there are zero-persistence launchers — `npx freellmapi launch` and `launch-codex` — that inject credentials into the child process only, so nothing lands in a config file. Note the base URLs differ: Claude Code points at the **root**, everything else at `/v1`.

Read the code

```
npm install && npm run dev   # server :3001, dashboard :5173, both with HMR
```

OmniRoute

github.com/diegosouzapw/OmniRoute

Install

```
npm i -g omniroute
# server boots on http://localhost:20128
```

It is designed to answer before you have added any keys — a keyless provider is pre-wired into the `auto` combo:

```
curl http://localhost:20128/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"auto","messages":[{"role":"user","content":"Hello!"}]}'
```

Docker and source installs are also available (`diegosouzapw/omniroute` on Docker Hub), plus an Electron desktop build in Releases. One thing to know: the project generates a local `.env` during npm install/postinstall. It is gitignored by design — never commit it, and rotate any secret that leaks.

Configure

Open `http://localhost:20128` for the dashboard and add your provider keys there (AES-256-GCM encrypted, local-first). The `/dashboard/free-tiers` page shows live used-versus-remaining per pool — the most useful screen for understanding what you actually have. Then either use `auto` and let it score providers live, or build a **combo**: an explicit chain of models with a routing strategy attached.

Model ID	Behaviour
auto	Balanced; sticks to the last known good provider
auto/coding	Quality-weighted for code generation
auto/fast	Lowest latency first
auto/cheap	Cheapest per token first
auto/offline	Most remaining quota / rate-limit headroom first

For agent development, `auto/coding` during the day and `auto/offline` once things start rate-limiting is a reasonable pattern. Nineteen routing strategies exist if you want to hand-build chains — priority, cost-optimized, round-robin, fusion, pipeline and so on.

Use from Python

Same shape as above — point the OpenAI SDK at `http://localhost:20128/v1`. Responses carry `X-OmniRoute-Decision`, naming the strategy, provider and latency that served the request.

Launch coding agents

```
omniroute run claude   --model openai/gpt-5.4
omniroute run codex    --model glm/glm-5.2
omniroute run aider    --model glm/glm-5.2
omniroute run opencode --model glm/glm-5.2

# or write the tool's own config interactively:
omniroute configure codex
```

`--dry-run` previews the exact env and args without executing, and `--api-key-env NAME` keeps secrets out of shell history.

What the gateway does that is hard to hand-roll

The value is not key storage. It is five pieces of boring, high-volume work:

- **Rate-limit bookkeeping.** Every provider counts differently — requests/min, tokens/min, requests/day resetting at UTC midnight, sometimes per-model rather than per-account. Staying under all of them at once needs a live counter per (provider, model, key) that decrements *before* you send, not after you are rejected.
- **Wire-format translation.** Gemini, Anthropic and Ollama do not speak OpenAI. Some providers return tool calls as plain text in the content field that must be parsed back into real tool_calls. Some silently ignore response_format.
- **Catalog drift.** Free tiers change weekly — models launch, get retired, quotas tighten silently. Both projects pull a signed catalog periodically; this is FreeLLMAPI's entire monetisation (free installs get a 30-day-delayed snapshot, $19/yr gets same-day).
- **Cooldown and circuit-breaking.** A naive retry loop hammers a rate-limited key and gets you flagged. Proper backoff, honouring Retry-After, isolating per-key from per-model from per-provider failures.
- **Stickiness.** Mid-conversation model switching breaks coherence badly. FreeLLMAPI pins a conversation to one model for 30 minutes and can inject a handoff note when a switch is unavoidable.

Manual versus framework

	Hand-rolled	Gateway
Account creation	You	You — identical
Provider ToS exposure	Yours	Yours — identical
Code in your app	Fallback logic everywhere	Change base_url; done
Adding a provider	Write an adapter	Paste a key
Extra moving part	None	A Node process + SQLite that can fail
Debuggability	Full	You now debug someone else's router
Maintenance	Yours, forever	Theirs, with a paid tier for freshness

For two or three providers, a hand-rolled try/except cascade with a dict of counters is maybe 150 lines and arguably better — you control it and there is no extra process. Past roughly ten providers it becomes its own project you have to maintain, and that is the point where these tools start paying for themselves.

Know the trade-offs

- **The ToS question is yours.** Both repos say so explicitly. Personal, single-user proxying of your own keys is generally tolerated; using stacked free tiers to serve an application's users is a different thing, and “the gateway did it” is not a defence.
- **No frontier models, and quality degrades through the day.** The strong models hit their daily caps first, so by evening auto falls through to weaker ones. An agent that worked at 10am can quietly get dumber at 6pm — brutal to debug if you are not watching the routing header.
- **Your prompts reach many companies' free tiers.** Free tiers are the ones most likely to carry training-on-your-data terms. Never route client or work data through this.
- **Variable latency, no SLA.** FreeLLMAPI's own README is blunt: free tiers are for prototyping, not a stable inference substrate. Swap in a paid API before you ship.

> **Keep an exit hatch**
>
> Put a USE_GATEWAY flag in your FastAPI settings so base_url swaps back to a paid Anthropic or Azure OpenAI endpoint with one environment variable. You do not want gateway-specific assumptions leaking into code that eventually deploys. If you want this pattern with production support, LiteLLM is the Python-native equivalent and the one to reach for in a real deployment.

Suggested path: start with FreeLLMAPI and five keys. Confirm X-Routed-Via is rotating the way you expect and that routed quality is good enough for your agent loops. Try OmniRoute afterwards if you want the compression engines or the wider catalog.